

Technical Specification — Black Fitness ระบบจองคลาส

เอกสารข้อกำหนดทางเทคนิค (Technical Specification) ฉบับนี้อธิบายสถาปัตยกรรม เทคโนโลยี การไหลของข้อมูล กฎทางธุรกิจ และข้อพิจารณาด้านความปลอดภัยของระบบ จองคลาสออกกำลังกาย Black Fitness

อ้างอิงจากซอร์สโค้ดจริงในโปรเจกต์ (Vue 3 + Firestore + LINE LIFF)

สารบัญ

- ภาพรวมระบบ
- สถาปัตยกรรม
- เทคโนโลยีที่ใช้
- โครงสร้างโปรเจกต์
- การยืนยันตัวตนและสิทธิ์
- การไหลของข้อมูลหลัก
- กฎทางธุรกิจ
- การจัดการ Concurrency
- ประสิทธิภาพและแคช
- ความปลอดภัย
- การ Deploy
- ข้อจำกัดและงานที่ควรทำต่อ

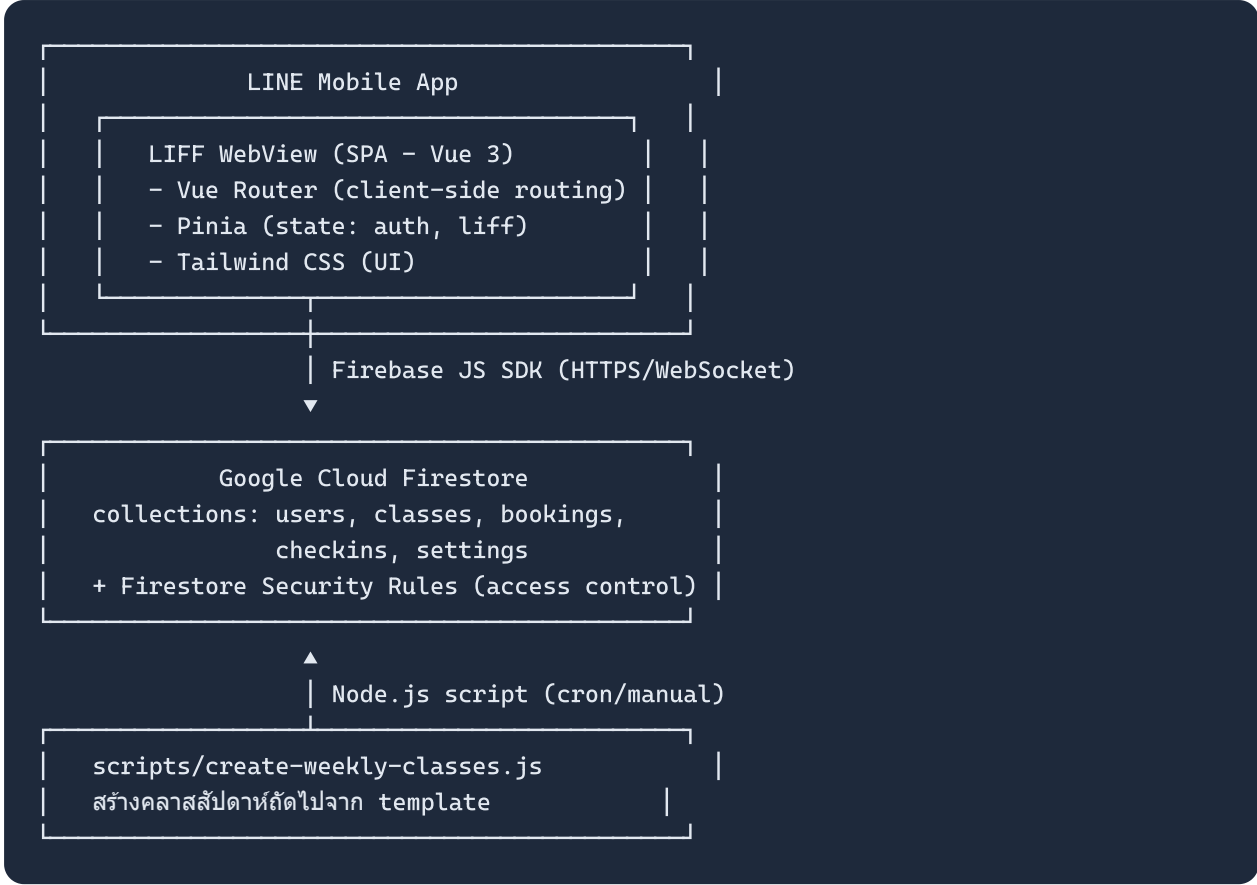
1. ภาพรวมระบบ

Black Fitness เป็น Web Application แบบ Single Page Application (SPA) ที่ทำงานภายใน LINE ผ่าน LINE Front-end Framework (LIFF) ให้บริการ:

- สมาชิกดูบัตรสมาชิกดิจิทัล จองคลาส ดู/ยกเลิกประวัติการจอง
- ผู้ดูแลระบบ (admin/staff) จัดการคลาส สมาชิก การจอง เช็किनด้วยการสแกน และตั้งค่าประเภทคลาส

ระบบเป็นแบบ **serverless** — Frontend เชื่อมต่อกับ Cloud Firestore โดยตรง ผ่าน Firebase JavaScript SDK ไม่มี backend server หรือ REST API กลาง การควบคุมสิทธิ์ทำผ่าน Firestore Security Rules

2. สถาปัตยกรรม



จุดสำคัญ:

- **Client-direct-to-DB:** ไม่มี API layer คั่นกลาง ลด latency และความซับซ้อน แต่ย้ายภาระความปลอดภัยไปไว้ที่ Security Rules ทั้งหมด
- **LIFF เป็นตัวกลางยืนยันตัวตน:** ใช้ LINE User ID เป็น primary key ของผู้ใช้
- **Batch job** แยกต่างหาก: การสร้างคลาสรายสัปดาห์เป็น Node script รันนอกแอป

3. เทคโนโลยีที่ใช้

ส่วน	เทคโนโลยี	เวอร์ชัน
Frontend Framework	Vue 3 (Composition API)	^3.4.0
Build Tool	Vite	^5.0.8
Routing	Vue Router	^4.2.5
State Management	Pinia	^2.1.7
UI / CSS	Tailwind CSS	^3.4.0
Database	Cloud Firestore (Firebase)	^10.7.1
LINE Integration	@line/liff	^2.23.1
Dialog/Alert	SweetAlert2	^11.26.2
Date Handling	date-fns (+ locale th)	^3.0.6

หมายเหตุ: ไฟล์ build (dist/) แสดงว่า production ใช้ Firebase v12 และ Vite v7 ขณะที่ client/package.json ระบุ v10/v5 — ควรตรวจสอบความสอดคล้องของเวอร์ชันก่อน deploy

4. โครงสร้างโปรเจกต์

```
black-classbooking/
├── client/
│   ├── src/
│   │   ├── views/                # หน้าหลัก (route components)
│   │   │   ├── Home.vue          # หน้าแรก + บัตรสมาชิก
│   │   │   ├── Booking.vue       # จองคลาส
│   │   │   ├── ClassDetail.vue   # รายละเอียดคลาส
│   │   │   ├── History.vue       # ประวัติการจอง
│   │   │   ├── ProfileSetup.vue  # ตั้งค่าโปรไฟล์ผู้ใช้ใหม่
│   │   │   ├── MemberCardView.vue # บัตรสมาชิก (แชร์ได้)
│   │   │   └── Admin.vue         # แผงผู้ดูแล (~2,700 บรรทัด)
│   │   ├── components/
│   │   │   ├── admin/           # DashboardStats, UserList, ClassList
│   │   │   ├── BottomNav.vue    # แถบเมนูล่าง
│   │   │   ├── MemberCard.vue   # การ์ดสมาชิก
│   │   │   ├── CheckinScanner.vue # สแกนเช็คอิน
│   │   │   └── DevTools.vue     # เครื่องมือ dev (mock user)
│   │   ├── stores/
│   │   │   ├── auth.js          # สถานะผู้ใช้ + บทบาท
│   │   │   └── liff.js          # สถานะ LIFF + mock profile
│   │   ├── utils/
│   │   │   ├── firestore.js     # helper (safeBooking, cache)
│   │   │   ├── cache.js         # ระบบแคช
│   │   │   ├── errorHandler.js  # จัดการ error เป็นภาษาไทย
│   │   │   └── dialog.js        # wrapper SweetAlert2
│   │   ├── constants/classTypes.js # ประเภท/คลาสย่อย + คำอธิบาย
│   │   ├── router/index.js      # เส้นทาง + route guards
│   │   └── firebase.js          # config Firebase
│   └── package.json
├── scripts/create-weekly-classes.js # สร้างคลาสรายสัปดาห์
└── firestore.rules                # Security Rules
```

5. การยืนยันตัวตนและสิทธิ์

5.1 การเข้าสู่ระบบ

- แอปเริ่มที่ `App.vue` → เรียก `liffStore.initLiff()`
- โหมด **production**: เรียก `liff.init()` แล้วดึง LINE profile (`liff.getProfile()`) ได้ `userId` , `displayName` , `pictureUrl`
- โหมด **dev**: ใช้ mock profile (กำหนด uid/ชื่อ/รูปผ่าน query param ได้ เช่น `?mockUid=Utest002&mockName=สมชาย`)
- `authStore.signInWithLineUserId()` ตรวจสอบเอกสารใน `users/{lineUserId}` :
 - ถ้าไม่มี → ตั้ง `needsProfileSetup = true` (ไปหน้า ProfileSetup)

- ถ้ามี → โหลดโปรไฟล์ กำหนดบทบาท (admin/staff/user)

5.2 บทบาท (Roles)

บทบาท	สิทธิ์
user	จองคลาส, ยกเลิกการจองของตัวเอง, ดูประวัติ/บัตรสมาชิกของตัวเอง
staff	สิทธิ์ผู้ดูแลแบบจำกัด: ตั้งวันหมดอายุ + แพ็คเกจสมาชิก + สแกนเช็คอิน
admin	สิทธิ์เต็ม: จัดการคลาส/สมาชิก/การจอง, ตั้ง cooldown, เปลี่ยน role, ตั้งค่าประเภทคลาส

Getter `canAccessAdmin = isAdmin || isStaff` ควบคุมการเข้าถึงหน้า `/admin`

5.3 Route Guards

`router/index.js` ตรวจสอบ meta ของแต่ละเส้นทาง:

- `requiresAuth` → ต้อง login แล้ว ไม่ขึ้นแดงไป `/`
- `requiresProfile` → ถ้า `needsProfileSetup` เแดงไป `/profile-setup`
- `requiresAdmin` → ถ้าไม่ใช่ admin/staff เแดงไป `/`

หมายเหตุทางเทคนิค: การ reload หน้าเต็ม (full page load) จะรีเซ็ต Pinia state ก่อนที่ auth จะ hydrate กลับจาก localStorage ทำให้ guard เแดงกลับ `/` ภายในแอปจึงใช้ client-side navigation เป็นหลัก

6. การไหลของข้อมูลหลัก

6.1 การลงทะเบียนผู้ใช้ใหม่

```
LIFF login → ไม่พบ users/{uid} → ProfileSetup.vue
→ กรอกข้อมูล (ชื่อเล่น, ชื่อจริง, เลขบัตรปชช. 13 หลัก + checksum,
เบอร์โทร, วันเกิด, เพศ, ปัญหาสุขภาพ, ผู้ติดต่อฉุกเฉิน, ยอมรับเงื่อนไข)
→ completeProfileSetup() → setDoc(users/{uid})
→ profileCompleted = true
```

การ validateฝั่ง client: ต้องเป็นภาษาไทยเท่านั้น, เลขบัตรประชาชนตรวจ checksum 13 หลักตามมาตรฐานไทย, เบอร์โทรต้องขึ้นต้น 0 และมี 10 หลัก

6.2 การจองคลาส

```
Booking.vue → เลือกวันที่ → loadClasses() (query classes ตาม date)
→ กดจอง → ตรวจสอบ canBook() → confirmBooking()
→ runTransaction:
  - อ่าน classes/{id}
  - ตรวจสอบ currentBookings < maxCapacity
  - สร้าง bookings/{auto-id} (status='confirmed')
  - update classes/{id}.currentBookings += 1
```

6.3 การยกเลิกการจอง

```
History.vue → canCancel() (ต้องเหลือ > 5 ชม. ก่อนคลาส)
→ confirmCancel()
→ updateDoc(bookings/{id}) status='cancelled', cancelledAt
→ updateDoc(classes/{id}) currentBookings -= 1
```

6.4 การเช็คอิน (Admin/Staff)

```
CheckinScanner → สแกน QR/ระบุ uid → ตรวจสอบ membership valid
→ addDoc(checkins/{auto-id}) บันทึก uid, membershipValid, classId, checkedInAt
```

6.5 การสร้างคลาสรายสัปดาห์ (batch)

```
node scripts/create-weekly-classes.js [--weeks N] [--dry-run]
→ ดึงคลาสสัปดาห์ปัจจุบัน → สร้าง template ตาม dayOfWeek
→ หาสัปดาห์ถัดไปที่ยังไม่มีคลาส
→ addDoc(classes) ทีละคลาส (ข้ามถ้ามี date+time+name ซ้ำ)
```

7. กฎทางธุรกิจ

กฎ	รายละเอียด	แหล่งอ้างอิง
จองล่วงหน้า	สูงสุด 14 วัน (Booking.vue) — หมายเหตุ ClassDetail.vue ใช้ 7 วัน ไม่ตรงกัน	<code>availableDates</code> loop 14 วัน
ปิดรับจองก่อนคลาส	ต้องเหลือ > 30 นาที ก่อนคลาสเริ่ม	<code>canBook()</code> <code>minutesDiff > 30</code>
ยกเลิกได้	ต้องเหลือ > 5 ชั่วโมง ก่อนคลาสเริ่ม	<code>canCancel()</code> <code>hoursDiff > 5</code>
ความจุคลาส	จองได้เมื่อ <code>currentBookings < maxCapacity</code> (default 15)	transaction check
จองซ้ำ	ห้ามจองคลาส+วันเดิมที่ยัง confirmed	<code>alreadyBooked</code> check
สมาชิกหมดอายุ	จองไม่ได้ถ้า <code>membershipExpiry ≤ now</code>	<code>isMembershipValid()</code>
Cooldown	admin ระบุวันที่จองชั่วคราว (<code>cooldownUntil</code>)	<code>isCooldownActive()</code>
แพ็คเกจ Gold	<code>memberType === 'gold'</code> จองคลาสไม่ได้	<code>canBook()</code>
สถานะการจอง	<code>confirmed</code> / <code>cancelled</code> / <code>completed</code>	History.vue statusMap

⚠ พบความไม่สอดคล้อง: หน้าต่างจองล่วงหน้าใน Booking.vue (14 วัน) กับ ClassDetail.vue (7 วัน) ต่างกัน ควร refactor ให้เป็นค่าคงที่กลางค่าเดียว

8. การจัดการ Concurrency

การจองใช้ **Firestore Transaction** (`runTransaction`) เพื่อป้องกัน race condition เมื่อมีผู้ใช้หลายคนจองคลาสเดียวกันพร้อมกัน:

- อ่าน `currentBookings` ภายใน transaction
- ตรวจความจุ และเพิ่มค่าด้วย `increment(1)` แบบ atomic
- ถ้าคลาสเต็มระหว่างนั้น transaction จะ throw `'คลาสเต็มแล้ว'`

การยกเลิก (History.vue) ใช้ `updateDoc` 2 ครั้งแยกกัน (ไม่ใช่ transaction) ซึ่งมีความเสี่ยงเล็กน้อยหากครั้งใดครั้งหนึ่งล้มเหลว — `utils/firestore.js` มี `safeCancelBooking()` แบบ transaction ไว้แล้วแต่หน้า History ยังไม่ได้ใช้

9. ประสิทธิภาพและแคช

- โหลดคลาสตามวัน: query `where('date', '==', ...)` โหลดเฉพาะวันที่เลือก ไม่ดึงทั้ง collection
- โหลด **bookings** ครั้งเดียว: ดึงตอน mount แล้ว optimistic update ใน memory หลังจอง/ยกเลิก ไม่ reload ทุกครั้ง
- ระบบแคช (`utils/cache.js`): เก็บผล query ใน localStorage พร้อม TTL ใช้ใน Admin dashboard (CACHE_KEYS) ลด Firestore reads
- การ sync รูปโปรไฟล์ทำแบบ inline ใช้ข้อมูลที่ได้มาแล้ว ไม่ยิง read เพิ่ม

10. ความปลอดภัย

10.1 Firestore Security Rules

ไฟล์ `firestore.rules` กำหนดสิทธิ์ระดับ collection:

- `users` : อ่านได้เฉพาะเจ้าของหรือ staff/admin; แก้ไขเองได้แต่ห้ามแก้ `role` , `membershipExpiry` , `cooldownUntil` (เฉพาะ admin); ห้ามลบ
- `classes` : อ่านได้ทุกคน; เขียนได้เฉพาะ admin
- `bookings` : อ่าน/สร้าง/ยกเลิกได้เฉพาะเจ้าของ (ต้อง membership valid); staff/admin จัดการได้ทุกอย่าง; ห้ามลบ
- `checkins` : สร้างได้เฉพาะ staff/admin; ห้ามแก้/ลบ
- `settings` : อ่านได้ทุกคน; เขียนได้เฉพาะ admin

10.2 ความเสี่ยงที่ต้องระวัง

⚠️ **สำคัญ:** Security Rules ปัจจุบันอ้างอิง `request.auth` (Firebase Auth) แต่ระบบใช้ LINE LIFF ยืนยันตัวตน **ไม่มี Firebase Auth** หาก rules นี้ถูก deploy โดยไม่มี Firebase Auth integration การเขียนข้อมูลทั้งหมดจะถูกปฏิเสธ ดูรายละเอียดใน SECURITY_MIGRATION.md — ต้องเลือกแนวทาง (เพิ่ม Firebase Custom Auth หรือปรับ rules) ก่อน deploy

- Firebase config (apiKey ฯลฯ) อยู่ใน client เป็นเรื่องปกติของ Firebase แต่ความปลอดภัยจริงต้องพึ่ง Security Rules ที่ถูกต้อง
- เลขบัตรประชาชนและข้อมูลสุขภาพเป็นข้อมูลอ่อนไหว (PDPA) — ต้องมั่นใจว่า rules จำกัดการอ่านเฉพาะเจ้าของและ staff/admin จริง

11. การ Deploy

```
# Frontend (Firebase Hosting)
cd client
npm run build          # vite build → dist/
npm run deploy         # build + firebase deploy --only hosting

# Security Rules (อ่าน SECURITY_MIGRATION.md ก่อน!)
firebase deploy --only firestore:rules

# สร้างคลาสรายสัปดาห์ (cron หรือ manual)
node scripts/create-weekly-classes.js --weeks 1
```

ตัวแปรแวดล้อม (client/.env):

- `VITE_LIFF_ID` — LIFF ID (default fallback มีใน `liff.js`)
- `VITE_USE MOCK_PROFILE` — เปิด mock profile นอกโหมด dev

12. ข้อจำกัดและงานที่ควรทำต่อ

1. **Firebase Auth** ยังไม่มี — Security Rules ต้องการ แต่ระบบใช้ LIFF ต้องตัดสินใจแนวทางก่อน production
2. หน้าต่างจองล่วงหน้าไม่ตรงกัน (14 วัน vs 7 วัน) — ควรรวมเป็นค่าเดียว
3. การยกเลิกไม่ใช่ **transaction** — ควรเปลี่ยนไปใช้ `safeCancelBooking()`
4. เวอร์ชัน **dependency** ไม่ตรงกัน ระหว่าง source กับ build output
5. ไม่มี **automated tests** — ควรเพิ่ม unit/integration test
6. **batch** สร้างคลาสรันด้วยมือ — ควรตั้ง cron/Cloud Scheduler

เอกสารนี้สร้างจากการวิเคราะห์ซอร์สโค้ดจริง ณ เวอร์ชันปัจจุบัน